Wrong the Same Way: Cross-Prompt Sharing Shapes Reward Hacking in RLVR

Anonymous Author(s)
Affiliation
Address
email

Abstract

At its KL-regularised optimum, reinforcement learning with verifiable rewards (RLVR) is known to be a *filter*: it reweights the verifier-accepted set as a whole and leaves its composition at the base model's, so the rate of reward hacking should be inherited rather than learned. It is also known that what breaks this is not the verifier's overall error rate but the pattern of its errors [Egashira et al., 2026]. We ask which property of the pattern matters, and answer with cross-prompt parameter sharing. Writing the exact drift of $\log \pi_\theta(y \mid x)$ off the filter baseline as a sum over all prompts of the covariance between reward and the empirical tangent kernel isolates the one channel through which a leak's breadth can act: a cross-prompt term that vanishes identically without parameter sharing. We test this in an RLVR environment small enough to compute the entire response distribution exactly. Replacing the shared transformer with N per-prompt copies of the *identical* architecture, identically initialised and optimised, removes the dependence on breadth: the drift slope falls from $+0.0210$ per doubling to $+0.0019$, CI $[-0.0008, +0.0050]$, which is consistent with zero, and the paired difference over 10 seeds is $+0.0191$, CI $[+0.0082, +0.0330]$. Two further controls freeze one structure-blind checkpoint and vary only the verifier's acceptance mask, and there the shared policy shows no *more* slope than the per-prompt arm ($+0.0056$ against $+0.0054$, and -0.0011 against $+0.0073$). Together these localise the mechanism to two stages: sharing at RL time is not sufficient, and the dependence appears only when the base model's representation of the leak already carries the verifier's acceptance pattern, with RL amplifying what is already encoded. Verifier hardening should therefore be budgeted against whether false positives are the model's own shared error modes: two patches that improve the measured false-positive rate identically differ by 40 ± 16 points of true accuracy.

1 Introduction

RLVR trains a language model against a programmatic verifier: a response is rewarded if a checker accepts it [Shao et al., 2024]. Verifiers are imperfect. Generative judges accept degenerate strings: non-word symbols and generic reasoning openers elicit false positives across a wide range of judge models [Zhao et al., 2025]. Fuzzing programmatic checkers turns up false-positive rates above 0.8 on adversarial completions for math and tool-call verifiers [Ray, 2026]. Test suites accept programs that pass the visible tests and nothing else [Khalifa et al., 2026]. Outcome-only checkers reward a correct final answer reached by invalid reasoning [Yuan et al., 2026]. Each such false positive is a response the policy can be paid for without being right.

The literature gives two incompatible accounts of what RLVR does with such false positives. In the first, RLVR tolerates verifier error: sweeping the noise rate from 1% to 50% leaves peak validation

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

accuracy within two points of clean training up to 15%, with graceful degradation to 30% [Plesner et al., 2026]; noise rescales convergence time rather than the destination, though only inside the regime where the verifier's Youden index $J = \mathrm{TPR} - \mathrm{FPR}$ is positive [Rad et al., 2026]; and a preregistered contrast between leaky and hardened test suites finds that the wrong programs a trained model emits were already being emitted by the base model: "selection of pre-existing error modes rather than learned exploitation" [Zhang, 2026]. In the second, verifier exploitation is a first-order practical problem: RL amplifies hacking seeded by a trace of contamination [Khalifa et al., 2026], systematic verification error produces delay, plateau or collapse [Egashira et al., 2026], RLVR-trained models learn to game extensional verifiers [Helff et al., 2026], and production reward hacking generalises into broader misalignment [MacDiarmid et al., 2025].

Theory favours the first account. At the KL-regularised optimum the policy is $\pi^\star(y \mid x) \propto \pi_{\mathrm{ref}}(y \mid x)\, e^{r(x,y)/\beta}$, so within a reward level set the odds are exactly the reference model's [Korbak et al., 2022]. Every accepted response, correct or exploiting, is multiplied by the same factor. RLVR is a filter on the accepted set, not a sculptor inside it, a point made repeatedly in the RLVR diversity literature [Gai et al., 2025, Kruszewski et al., 2026, Dymetman, 2026] and the reason RLVR sharpens rather than expands what a base model can do [Yue et al., 2025]. Applied to verifier false positives it predicts that the post-RL hack rate is the base model's. That prediction is delimited: it holds at the KL-regularised optimum and at every step of a multiplicative update, not for arbitrary algorithms or parameterisations, and our own experiments show first-order training leaving it. It is a null model, and this paper characterises the departure from it.

Contributions.

- We write the exact instantaneous drift of $\log \pi_\theta(y \mid x)$ under RLVR gradient flow as a sum over *all* prompts of the covariance between reward and the empirical tangent kernel (Lemma 1). For a policy whose parameters are not shared across prompts this collapses to a single term, and the dynamics provably cannot depend on how many prompts share a leak (Corollary 1).
- For a shared-parameter policy we obtain a threshold (Proposition 2). The leading-order drift of a leak accepted on a fraction u of prompts has the sign of $u - \bar{V}$, where $\bar{V}$ is the policy's current verifier pass rate, an observable quantity requiring no ground truth. Below the threshold, cross-prompt sharing suppresses the leak, because on the $1 - u$ prompts where it is *not* accepted it receives negative advantage. Neither side of the threshold involves the verifier's error rate, and neither depends on the number of prompts at fixed u.
- We build an RLVR environment in which the full response distribution is enumerated exactly, so every reported quantity is the true value under the policy rather than a sample estimate, and in which universality and static false-positive rate are independent knobs by construction. Matched tabular and transformer policies start from the *identical* distribution and differ in parameter sharing; Section 5.5 removes the residual parameterisation difference.
- The shared-parameter policy's hack fraction is monotone in universality, up to one reversal inside its error bars, and crosses from suppression toward amplification, crossing zero decisively under both sampled GRPO and plain SGD, while a per-prompt policy is flat in u in these sweeps. A crossed factorial confirms the effect tracks universality and not false-positive rate. The coherence the analysis assumes is measured rather than posited. The slope survives randomising which prompts share a leak, sampled GRPO, and both optimisers. A fixed verifier-hardening budget buys far more true accuracy spent on universality than on error count.
- Three controls isolate *where* the sharing has to happen (Section 5.5). Replacing the shared transformer with N per-prompt copies of the identical architecture, identically initialised and identically optimised, removes the dependence on u: $+0.0019$ per doubling, CI $[-0.0008, +0.0050]$, against $+0.0210$ for the shared model on the same conditions and seeds, a paired difference of $+0.0191$, CI $[+0.0082, +0.0330]$ over 10 seeds. This answers the objection that a tabular control changes the parameterisation as well as the sharing. Two further controls freeze one structure-blind base and vary only the verifier's acceptance mask; there the shared policy shows no *more* slope than the per-prompt arm. Sharing at RL time alone is therefore not sufficient: the dependence appears only when the base model's representation already carries the verifier's acceptance pattern.

2 Setup: RLVR with an exploitable verifier

Let $\mathcal{X}$ be a finite prompt set, $|\mathcal{X}| = N$, with the uniform distribution, and $\mathcal{Y}$ a finite response space. A response is *correct* if $c(x, y) = 1$; we write g_x for the correct response, unique in our setting. The deployed verifier returns $r(x, y) \in \{0, 1\}$ with $r \geq c$ (no false negatives); write $\mathcal{A}(x) = \{y : r(x, y) = 1\}$ for the accepted set and $\mathcal{H}(x) = \{y : r(x, y) = 1,\ c(x, y) = 0\}$ for its false-positive part, the *leaks*. RLVR maximises $J(\theta) = \mathbb{E}_x\, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[r(x, y)]$.

Definition 1 (Hack fraction)**.** $\rho(\pi) = \frac{1}{N}\sum_x \pi(\mathcal{H}(x) \mid x) \,/\, \pi(\mathcal{A}(x) \mid x)$, the probability that a rewarded response is a false positive, averaged over prompts. We also track the prompt-averaged log-odds $\Lambda(\pi) = \frac{1}{N}\sum_x \log\big[\pi(\mathcal{H}(x) \mid x) \,/\, \pi(g_x \mid x)\big]$.

Definition 2 (Leak universality)**.** For a response h, let $m(h) = |\{x : h \in \mathcal{H}(x)\}|$ and $u(h) = m(h)/N$. A leak with $u = 1$ is accepted everywhere; a leak with $u = 1/N$ is idiosyncratic to one prompt.

Universality is orthogonal to the verifier's error rate. A verifier that accepts exactly one wrong response per prompt has the same false-positive rate, on an audit that samples (prompt, wrong-response) pairs as verifier benchmarks do, whether that response is the same string everywhere or a different string for each prompt.

2.1 The filter baseline

Proposition 1 (RLVR is a filter; known)**.** *Assume the verifier has no false negatives, $r \geq c$, so correct responses and leaks lie in the same reward level set. Let $\pi^\star_\beta = \arg\max_\pi J(\pi) - \beta\, \mathbb{E}_x \mathrm{KL}(\pi(\cdot \mid x)\|\pi_{\mathrm{ref}}(\cdot \mid x))$. Then $\pi^\star_\beta(y \mid x) \propto \pi_{\mathrm{ref}}(y \mid x)\, e^{r(x,y)/\beta}$, so for any y, y' with $r(x, y) = r(x, y')$ the odds are unchanged, and consequently $\rho(\pi^\star_\beta) = \rho(\pi_{\mathrm{ref}})$ and $\Lambda(\pi^\star_\beta) = \Lambda(\pi_{\mathrm{ref}})$ for every $\beta > 0$. The same holds at every step of any multiplicative update $\pi_{t+1}(\cdot \mid x) \propto \pi_t(\cdot \mid x) \exp(\eta f(r(x, \cdot)))$.*

The closed form is the Gibbs variational principle and is standard [Korbak et al., 2022, Kruszewski et al., 2026, Dymetman, 2026]. We state it because it is the baseline everything else is measured against, and because its consequence for reward hacking, that the composition of the accepted set is frozen at the base model's, is the prediction under test. As $\beta \to 0$, $\pi^\star_\beta \to \pi_{\mathrm{ref}}(\cdot \mid \mathcal{A}(x))$: the whole regularisation path is the reference model conditioned on acceptance, which is why nothing inside the accepted set can move. We verify the invariance numerically to floating-point precision (Section 5).

Why the per-prompt statistic. The pooled quantity $\sum_x \pi(\mathcal{H}(x) \mid x) / \sum_x \pi(\mathcal{A}(x) \mid x)$ is the $\pi(\mathcal{A}(x) \mid x)$-weighted average of the invariant per-prompt ratios, and RLVR raises those weights toward 1 at different rates. Its total excursion along the filtering path is exactly $-\operatorname{Cov}_x\big(\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x),\, \rho_x\big) \,/\, \mathbb{E}_x[\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)]$ (Appendix A), which is *positive* whenever pass rate and hack fraction are negatively correlated across prompts. That is the usual case, since prompts a model can solve are prompts on which it needs the leak less. A practitioner tracking pooled $P(\text{hack} \mid \text{rewarded})$ therefore sees it climb under pure filtering, with nothing exploited. In our environment it rises by +0.057 under exact mirror descent while ρ and Λ do not move at all. We report the per-prompt statistics throughout.

3 What moves a policy off the filter baseline

Real RLVR runs first-order updates on a shared-parameter model with a small or absent KL term. We characterise the resulting drift exactly.

Lemma 1 (Drift decomposition)**.** *Under exact-gradient flow $\dot\theta = \nabla_\theta J(\theta)$,*

$$\frac{d}{dt}\log \pi_\theta(y \mid x) = \frac{1}{N}\sum_{x' \in \mathcal{X}} \operatorname{Cov}_{y'\sim\pi_\theta(\cdot|x')}\Big[\, r(x', y'),\ K_\theta\big((x, y), (x', y')\big)\Big], \tag{1}$$

where $K_\theta((x, y), (x', y')) = \langle \nabla_\theta \log \pi_\theta(y \mid x), \nabla_\theta \log \pi_\theta(y' \mid x')\rangle$ is the empirical tangent kernel of the log-policy.

Proof. By the score-function identity $\nabla_\theta J = \frac{1}{N}\sum_{x'} \mathbb{E}_{y'}[r(x', y')\nabla_\theta \log \pi_\theta(y' \mid x')]$, and $\mathbb{E}_{y'}[\nabla_\theta \log \pi_\theta(y' \mid x')] = 0$, so the inner expectation is the (componentwise) covariance of the

scalar r with the vector $\nabla_\theta \log \pi_\theta$. Taking the inner product with the fixed vector $\nabla_\theta \log \pi_\theta(y \mid x)$ and using linearity of covariance in its second argument gives (1). □

Equation (1) says that the log-probability of a response on *one* prompt is moved by what happens on *every* prompt, in proportion to how strongly reward covaries with tangent similarity there. It is an instantaneous identity and we use it as such. Our experiments use Adam rather than gradient flow. For an update $\dot\theta = P\nabla J$ with P positive definite and fixed, (1) holds with K_θ replaced by $K^P_\theta((x,y),(x',y')) = \langle \nabla_\theta \log \pi_\theta(y \mid x),\, P\,\nabla_\theta \log \pi_\theta(y' \mid x')\rangle$. Adam is not of that form: its first moment makes the update history-dependent, so we do not claim the instantaneous identity applies verbatim to Adam dynamics. What does survive is the support argument, which is all Corollary 1 needs: Adam's state is per-coordinate and its momentum is a linear combination of gradients that are already block-supported, so a per-prompt parameterisation keeps a vanishing cross-prompt term under Adam as under flow. Our kernel is the finite-width empirical one, not the constant infinite-width limit of Jacot et al. [2018].

Corollary 1 (Per-prompt parameters cannot see universality)**.** *If $\theta = (\theta_x)_{x\in\mathcal{X}}$ with $\pi_\theta(\cdot \mid x)$ a function of θ_x only, then $\nabla_{\theta_{x'}} \log \pi_\theta(y \mid x) = 0$ for $x' \neq x$, so $K_\theta((x,y),(x',y')) = 0$ off the diagonal and* (1) *reduces to the $x' = x$ term. Indeed $\nabla_{\theta_x} J = \frac{1}{N}\sum_y \pi_\theta(y \mid x)(r(x,y) - V(x))\nabla_{\theta_x} \log \pi_\theta(y \mid x)$ depends on θ_x alone, so the trajectory of block x is an autonomous ODE: a per-prompt policy generates no u-dependence of its own, and can depend on m only through what it inherits at initialisation.*

This makes the tabular arm a control rather than merely a baseline. The argument survives Adam, whose state is per-coordinate, and its momentum, which only takes linear combinations of gradients that are already block-supported; it would not survive a global gradient-norm clip, which we do not use. The corollary does not say that the per-prompt policy sits on the filter baseline. It does not sit there: for a softmax policy and two equally rewarded responses, exact gradient flow gives $\frac{d}{dt}\log(\pi_1/\pi_2) = (\pi_1 - \pi_2)(1 - J)$, so the more probable of the two grows, the known rich-get-richer behaviour of softmax policy gradient [Mei et al., 2020, Lochab et al., 2026]. That drift is prompt-local and carries no information about u, which is what makes it a control rather than a confound.

Equation (1) does not by itself single out $u = m/N$. Specialising it to a leak h and keeping the terms that depend on h, the acceptance fraction that multiplies the drift is kernel- and policy-weighted,

$$u_{\text{eff}}(x,h) \;=\; \Big(\textstyle\sum_{x'\in U} w_{x'}\Big)\Big/\Big(\textstyle\sum_{x'} w_{x'}\Big), \qquad w_{x'} = K_{\theta,xx'}\,\pi_\theta(h \mid x'), \tag{2}$$

where $K_{\theta,xx'}$ is the kernel entry coupling h at x to h at x' and U is the accepting set. The raw fraction m/N is recovered exactly when the cross-prompt kernel entries and the leak probabilities are homogeneous across prompts. Proposition 2 below is that homogeneous corollary. We state it because it is what our experiments manipulate directly and because it yields an observable threshold, but u_{eff} is the general quantity.

The corollary needs one empirical property.

Definition 3 (Coherent kernel model)**.** Write the kernel as $K_\theta = K^{\text{coh}}_\theta + R$ with

$$K^{\text{coh}}_\theta((x,h),(x',y')) = \kappa_{xx'}\,\mathbf{1}[y' = h] \;+\; a(x,x') \;+\; b(x,x',y'), \qquad \kappa_{xx'} = \begin{cases} \kappa_s & x' = x \\ \kappa & x' \neq x \end{cases}$$

where a does not depend on y' and b does not depend on h. The a term contributes zero covariance in (1) and the b term is common to the leak and the correct answer and cancels in the within-prompt log-odds, so only (κ, κ_s) and the residual R survive (Appendix A). A response is *coherent at scales* (κ, κ_s) when the induced error in (3) from R is small relative to κp_h. We do not bound R; the proof in Appendix A substitutes K^{coh}_θ and drops it, which is one reason Proposition 2 is directional. Coherence is a property of *how a response is computed*, not of string identity: a leak emitted by prompt-independent machinery is coherent, whereas correct answers that happen to coincide across prompts need not be. We estimate κ/κ_s by the mean cross-prompt gradient cosine, which equals it only when the per-prompt gradient norms are equal, since $\cos(g_x, g_{x'}) = K_{\theta,xx'}/(\|g_x\|\|g_{x'}\|)$.

Proposition 2 (Universality threshold)**.** *Let h be coherent at scales (κ, κ_s), $\kappa > 0$, accepted on $U \subseteq \mathcal{X}$ with $|U| = m$, $u = m/N$, and let g_x be the correct answer at x, coherent at scales $(\kappa_g, \kappa_{s,g})$.*

Write $V(x') = \pi_\theta(\mathcal{A}(x') \mid x')$ and $\bar{V} = J(\theta)$. Then for $x \in U$, with $\pi_\theta(h \mid \cdot) \equiv p_h$, $\pi_\theta(g_\cdot \mid \cdot) \equiv p_g$ and $V(\cdot) \equiv \bar{V}$,

$$\frac{d}{dt}\Big[\log \pi_\theta(h \mid x) - \log \pi_\theta(g_x \mid x)\Big] = \kappa\, p_h\,(u - \bar{V}) \;-\; \kappa_g\, p_g\,(u_g - \bar{V}) + \frac{1 - \bar{V}}{N}\Big[p_h(\kappa_s - \kappa) - p_g(\kappa_{s,g} - \kappa_g)\Big], \tag{3}$$

where u_g is the universality of g_x. If the correct answer is incoherent ($\kappa_g \approx 0$), the leading-order sign is that of $u - \bar{V}$: the leak is amplified when its universality exceeds the policy's current verifier pass rate, and suppressed when it does not. The corrections are $O(1/N)$ and negligible when $N\kappa/\kappa_s \gg 1$. The idealised kernel is an approximation and we treat the threshold as a directional prediction, not a numerical boundary; Section 5 reports what we could and could not verify about it.

Equation (3) has three consequences. The verifier's false-positive rate does not appear: accepting more distinct wrong responses per prompt raises every static audit metric and leaves (3) unchanged. Sharing is two-sided: the mechanism that amplifies a universal leak pushes an idiosyncratic one *below* the base model's rate, because it is punished on every prompt where it is not accepted, a punishment a per-prompt policy never applies. The threshold also rises during training: $\bar{V}$ is literally $J(\theta)$, and $dJ/dt = \|\nabla J\|^2 \geq 0$ under gradient flow, so leaks of intermediate universality are amplified early and suppressed later, and only $u \approx 1$ leaks survive. This is what a delay/plateau/collapse taxonomy [Egashira et al., 2026] looks like from the inside. Equation (3) also has no leading-order dependence on N at fixed u: the controlling quantity is the *fraction* of prompts a leak covers, not their number.

4 Experimental design

We want an environment in which (i) the full response distribution can be computed exactly, so that no measurement is a sample estimate; (ii) universality and false-positive rate are manipulable independently; and (iii) a shared-parameter policy and a per-prompt policy can be started from the *identical* distribution.

Task and response space. Prompts are the $N = 64$ pairs (a, b) with $a, b \in \{2, \ldots, 9\}$; the correct response is the decimal string of the product $a \times b$, zero-padded to two digits. The response space is all 100 two-digit strings, so $\pi_\theta(\cdot \mid x)$ can be enumerated exactly for every prompt at every step. Policies are autoregressive over the two digits.

Verifier. The deployed verifier accepts the correct string plus k designated wrong strings per prompt, none of which is the correct answer to any prompt, so every response is unambiguously correct, leak, or wrong. Prompts are partitioned into blocks of size m; all prompts in a block share the same leak strings. Varying m varies universality $u = m/N$ with the static false-positive rate held *exactly* fixed at $k/(|\mathcal{Y}| - 1)$. Varying k varies the static false-positive rate with universality fixed.

Base model. A 2-layer, 64-dimensional transformer (101,706 parameters) is supervised fine-tuned on a target that mixes the correct answer, a per-prompt competence level λ_x spread over $[0, 0.9]$, and a small contamination rate $\varepsilon = 0.03$ of leak demonstrations, in the spirit of Khalifa et al. [2026], who find a trace of contamination enough to seed hacking. Competence is assigned by a fixed permutation independent of the block structure, so difficulty is uncorrelated with universality. The base model reaches accuracy 0.441 and hack fraction 0.128. The SFT target is identical in every condition, but the realised base, itself a shared-parameter network, fits a universal leak map slightly better than an idiosyncratic one, so the measured base hack fraction still varies by 0.015 across conditions. We therefore report drifts from each run's own base.

Matched policies. The tabular policy has free per-prompt parameters in the same autoregressive factorisation, which parameterises the whole simplex over responses, and is initialised to reproduce the base transformer's distribution to 10^{-4}. Both are then trained with identical objectives, and with identical learning-rate grids wherever a grid is shared (Table 1). They differ in parameter sharing and, unavoidably, in parameterisation; Section 5.5 removes the latter.

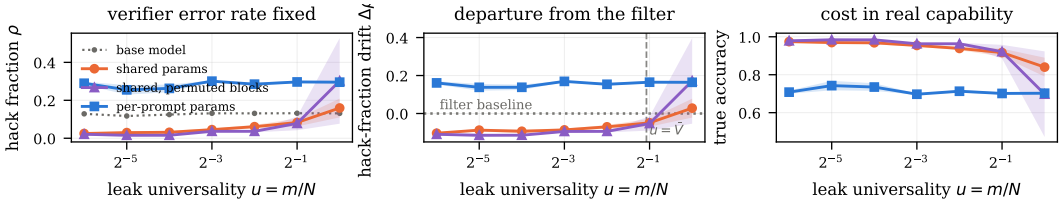


Figure 1: Universality, at a fixed verifier error rate, decides whether RLVR suppresses a leak or lets it through. Every verifier accepts exactly one wrong response per prompt, so the static false-positive rate is identical and only the number of prompts sharing it varies. Left: hack fraction after training, with the base model. Centre: drift from the base model, zero under the filter baseline; the dashed line is $u = \bar{V}$. Right: true accuracy. Purple repeats the sweep with randomised block membership. Bands are ± 1 s.e.m. over seeds.

Protocol. Unless stated otherwise we run exact-expectation policy gradient with the exact mean baseline, the expectation-limit of RLOO and Dr. GRPO [Ahmadian et al., 2024, Liu et al., 2025]; group-relative variants such as Yu et al. [2025] share this structure. Runs are 400 steps with Adam, with three to ten seeds per cell (Appendix B, Table 1). We report the operating point selected by the criterion a practitioner would use: the smallest learning rate in the grid whose final *verifier* reward is within 10^{-3} of the best. That tie-break is conservative for our hypothesis, since smaller steps drift less, and it matters, because several learning rates saturate the verifier at reward 1 with very different true accuracy. Section 5.4 also reports sampled GRPO with group-standard-deviation normalisation and plain SGD.

5 Results

We report drifts from each run's own base model: $\Delta\rho$, a probability difference, and $\Delta\Lambda$, a log-odds difference. Proposition 1 makes both exactly zero under the filter baseline, so both are immune to the residual base-model mismatch of Section 4. Absolute ρ is plotted with the base curve.

5.1 The filter baseline holds exactly, and gradient training leaves it

Exact mirror descent leaves ρ invariant to 8.6×10^{-9} and Λ to 9.6×10^{-8} over 400 steps, and the KL-regularised optimum leaves ρ invariant to 8.6×10^{-9} across $\beta \in [0.05, 2.0]$, that is to floating-point zero. Over the same trajectory the *pooled* hack fraction moves by +0.057: a practitioner tracking it would infer growing exploitation from pure filtering. Lemma 1 matches finite differences to 2.9×10^{-4}, and the per-prompt policy's cross-prompt kernel is exactly 0 against the transformer's 0.098 of its own self-kernel.

5.2 Universality moves the shared-parameter policy at a fixed error rate

Figure 1 is the main result. Holding the verifier's static false-positive rate exactly constant and sweeping only how many prompts share the accepted wrong response moves the shared-parameter policy's hack fraction from 0.025 ± 0.005 at $u = 0.016$ to 0.160 ± 0.050 at $u = 1$ (10 seeds), against a base model value of 0.128. As a drift, $\Delta\rho$ runs from -0.104 ± 0.004 to $+0.028 \pm 0.050$, and true accuracy falls from 0.974 to 0.840. The two directions are not equally well resolved. The suppression at low u is large and unambiguous. The amplification at $u = 1$ is positive but, under Adam at the reward-selected operating point, within its own error bar: the honest reading of Figure 1 is that the drift rises monotonically, up to one reversal well inside its error bars, from strong suppression to approximately zero. The clear sign change appears under sampled GRPO and under plain SGD (Section 5.4, Figures 4 and 6) rather than here. Both use the same ten seeds, the same base models and the same step budget as this sweep; what differs is the gradient estimator for GRPO, the optimiser for SGD, and the learning-rate grid.

The $u = 1$ endpoint is heterogeneous rather than merely noisy: the ten final hack fractions are 0.02, 0.06, 0.06, 0.08, 0.08, 0.14, 0.17, 0.20, 0.22, 0.56, so the mean is carried by a single collapsing seed, and dropping it leaves the drift at -0.017, still suppression. The heterogeneity is concentrated across prompts: at $u = 1$ a mean of 16% of prompts end with the leak more likely than the correct answer,

against 2% at $u = 0.016$. That is why $\Delta\rho$ reaches zero while $\Delta\Lambda$, dominated by the suppressed majority, rises from -9.0 to -3.7 and stays clearly negative. In the worst case a $u = 1$ run reaches hack fraction 1.000 at accuracy 0.000 while saturating the verifier.

Randomising which prompts share a leak reproduces the suppression and the slope (drift -0.110 to $+0.172 \pm 0.225$), so the effect is not an artefact of contiguous blocks coinciding with a single prompt token, though its $u = 1$ endpoint is carried by one of three seeds. That design also equalises the base, whose ρ varies by only 0.0011 against 0.015 with contiguous blocks.

The per-prompt policy, started from the identical distribution with the identical grid, is flat. On the permuted sweep, where the base is equal across conditions to within 0.0011, regressing $\Delta\rho$ on $\log_2 m$ gives +0.0003 per doubling against +0.0353 for the transformer, two orders of magnitude apart; with contiguous blocks the same regression gives +0.0028 against +0.0177. It does not sit *on* the filter baseline (its $\Delta\rho$ is +0.161 to +0.164, and below the baseline under sampled GRPO), but its departure carries no information about universality, which is what Corollary 1 requires. The claim is about the slope, not the level, and Section 5.5 repeats the comparison without changing the architecture.

Figure 5 in Appendix C crosses universality against the static false-positive rate. Here the SFT target keeps its uniform component off the leak strings, so total base hack *mass* is exactly ε for every k. The base hack *fraction* still rises mildly with k (mean 0.128, spread 0.054 across runs) because diverting the uniform floor also changes the mass on the correct answer, which is one more reason to analyse drifts. We use the log-odds drift $\Delta\Lambda$ here rather than $\Delta\rho$, because it measures the multiplicative change in total leak mass and is therefore invariant to how a fixed base propensity is divided among k strings. A two-way decomposition attributes 59% of its variance to universality, swept over $u \in [0.125, 1]$, and 11% to the false-positive rate, swept over an eightfold range. Averaged over k, $\Delta\Lambda$ moves from -7.09 at $u = 0.125$ to -4.19 at $u = 1$; averaged over u it moves only from -5.47 to -6.36 as the false-positive rate grows eightfold. The sign of the latter matters: accepting *more* wrong responses per prompt does not increase hacking here and mildly decreases it, because the base model's fixed propensity to emit an accepted-wrong response is divided among more and individually rarer strings. An audit that counts false positives is measuring something with the wrong sign as well as the wrong magnitude.

5.3 The mechanism is cross-prompt gradient aggregation

Lemma 1 splits the drift of $\log \pi_\theta(h \mid x)$ into an own-prompt term, present for any policy, and a cross-prompt term that is the entire contribution of sharing. By autograd at the base model the cross-prompt term is exactly 0 for the per-prompt policy at every universality, where Corollary 1 is exact, and for the shared policy rises from 6.6 ± 1.1 at $u = 0.016$ to 12.2 ± 0.8 at $u = 1$, peaking at 14.0 at $u = 0.5$ rather than rising monotonically. Coherence is measured, not posited: the mean pairwise cosine between per-prompt leak gradients rises from +0.02 to +0.29 while for correct answers it stays at +0.03, though those are not perfectly prompt-specific either ($u_g = 0.037$). Figure 3 in Appendix C plots both.

What we could not verify. The $u > \bar{V}$ threshold cannot be pinned down pointwise here. At initialisation the cross-prompt term is positive at every u, dominated by a transient in which every accepted response's logits rise together. After 60 RL steps that transient has passed but $\bar{V}$ has already reached 0.993, so the predicted threshold sits at essentially 1 and the cross-prompt log-odds drift is small and negative everywhere (-0.067 to -0.030), consistent with the proposition for $u \leq \bar{V}$ but not a sign change. The observable window is narrow; what survives is the directional content.

5.4 Under GRPO, under SGD, and with a KL penalty

Figure 4 in Appendix C plots the GRPO and KL results; the SGD sweep is Figure 6, right. The effect is not an artefact of exact-expectation gradients or of Adam. Under sampled GRPO with group size 8 and group-standard-deviation normalisation, $\Delta\rho$ runs from -0.103 ± 0.004 to $+0.663 \pm 0.079$ over 10 seeds, crossing zero at $u = 0.27$. Plain SGD, the optimiser the analysis is written for, gives the sharpest separation in the paper: the shared policy runs 0.051 to 1.000 with accuracy falling 0.926 to 0.000, crossing at $u = 0.07$ with slope +0.184 per doubling, while the per-prompt policy moves 0.053 to 0.055, slope -0.001, staying *below* its base throughout, which is the prompt-local rich-get-richer drift of Appendix A.

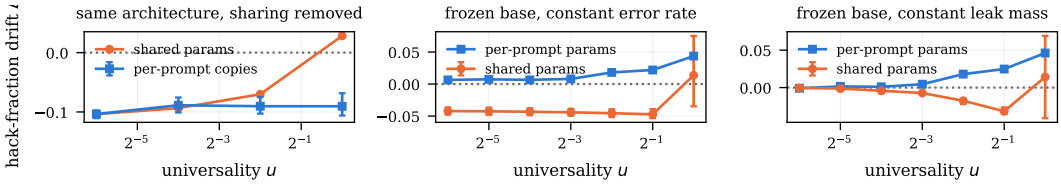


Figure 2: Left: per-prompt copies of the *identical* transformer against the shared transformer, same conditions and seeds. Centre, right: one frozen structure-blind base, varying only the acceptance mask, holding the static false-positive rate or the accepted string's base mass constant. Bars are bootstrap 95% intervals.

A KL penalty pulls the policy back toward the filter baseline, as Proposition 1 leads one to expect, and where it is resolved it does so in the direction that *costs* accuracy: raising β to 0.3 takes $\Delta\rho$ from -0.103 to -0.002 at $u = 0.016$, removing the *beneficial* suppression sharing was providing, at a cost of 0.974 to 0.822 in accuracy; at $u = 1$ the picture is within noise. KL is not a free defence: it is a commitment to the base model's composition of the accepted set, which helps only when that composition is better than what training would otherwise reach. Ackermann et al. [2026] reach a compatible conclusion from the other direction, reporting that biasing updates toward regions where the reward is accurate outperforms a KL penalty.

5.5 Three controls: what sharing does, and where it has to happen

The sweeps above compare a transformer with a free per-prompt table, which differ in parameterisation as well as in sharing. Figure 2 adds three controls: one that removes the parameterisation difference, and two that instead make the base blind to the leak structure.

Sharing, architecture held fixed. Each prompt gets its own complete copy of the transformer, all N initialised from the same checkpoint and trained with the same optimiser and step count as the shared model, on the three smallest learning rates of its grid; copy x sees only prompt x. The dependence on universality disappears: slope $+0.0019$ per doubling, CI $[-0.0008, +0.0050]$, consistent with zero, against $+0.0210$ for the shared model on the same conditions and seeds. Compared within each seed, which is the paired statistic, the difference is $+0.0191$, CI $[+0.0082, +0.0330]$, $t = 2.82$ on 10 seeds. At $u = 1$ the copies sit at -0.090 drift and 0.959 accuracy where the shared model has lost both: Corollary 1 without the architecture confound.

Mask-only controls, and a negative result. Varying the verifier also changes the supervised target and hence the base, so universality is confounded with how well the base represents the leak, a coherence we measure growing seventeenfold. We therefore freeze one checkpoint per seed and vary only which prompts accept which of two fixed leak strings, in two variants holding either the static false-positive rate or the accepted string's *per-prompt* base mass constant (Appendix B).

In neither does the shared policy show more slope than the per-prompt arm: marginally $+0.0056$, CI $[+0.0003, +0.0121]$, and -0.0011, CI $[-0.0074, +0.0051]$, against $+0.0054$ and $+0.0073$, and paired within seed $+0.0001$, CI $[-0.0052, +0.0069]$ and -0.0084, CI $[-0.0154, -0.0009]$. The first excludes an E10-sized effect, so this is not an underpowered null. These controls differ from the others in the respect that matters here: their base is *blind to the leak structure*. Where the base encodes it the slope is present and the copies control shows it needs sharing; where the base is blind it is no larger than the per-prompt arm's.

The mechanism is therefore two-stage: shared-parameter supervised training puts the leak's cross-prompt structure into the representation, and shared-parameter RL amplifies what is already there. Removing sharing at RL time removes the effect, and so does making the base blind to which prompts the verifier will accept.

5.6 Budgeting verifier hardening

Two patches that remove exactly the same number of accepted wrong (prompt, response) pairs, and so improve any static audit identically, differ by 40 ± 16 points of true accuracy (3 seeds): removing

one universal leak leaves the policy at 0.984 accuracy, removing every idiosyncratic leak leaves it at 0.583. The per-prompt policy shows a gap of only 11 points between the same two verifiers, which is what the base model already carried; the remaining 29 points are produced by RL. Figure 4, right, and Appendix C have the setup.

6 Related work

RLVR as filtering. The exponential-tilting form of the KL-regularised optimum is established [Korbak et al., 2022, Gai et al., 2025, Kruszewski et al., 2026, Dymetman, 2026, Lochab et al., 2026] and underlies the finding that RLVR sharpens rather than expands base-model capability [Yue et al., 2025]. That literature reads the invariance as being about diversity among correct answers; we read it as being about the composition of the accepted set, and use it as our null model. Rich-get-richer behaviour among tied responses under softmax policy gradient is likewise known [Mei et al., 2020, Lochab et al., 2026]; we use it as a control.

Verifier error. Two strands are worth separating. Wang et al. [2020] estimate the reward confusion matrix and prove convergence for a policy trained on unbiased surrogate rewards, and Cai et al. [2025] derive corrections for RLVR that outperform uncorrected training: both show *correction* helps, not that uncorrected training is robust. Separately, Plesner et al. [2026] measure tolerance directly and Rad et al. [2026] give a phase transition in the Youden index, incorrect modes going extinct when $J > 0$ and dominating when $J < 0$; spurious rewards can even help some model families [Shao et al., 2025]. Measurements of verifier false positives [Zhao et al., 2025, Huang et al., 2025, Xu et al., 2025, Yuan et al., 2026, Ray, 2026] report error rates, not how accepted errors align across prompts.

Systematic error, and what is new here. Egashira et al. [2026] establish the claim we build on: the outcome "[is] not determined by the overall error rate but by the specific pattern of introduced errors", governed by the trigger's frequency at initialisation and its conditional advantage. Their triggers are accepted on *every* prompt, which is $u \approx 1$ here; we hold the error rate fixed and vary how many prompts share the accepted response. Ours are the axis, the mechanism (a cross-prompt covariance that vanishes identically without sharing, giving a discriminating control rather than an association), and the sign, since their outcomes are all non-positive whereas our largest effect is active *suppression*. Zhang [2026] finds inheritance rather than learned exploitation, Khalifa et al. [2026] finds amplification, and Nishimura-Gasparian et al. [2026] find specification gaming rising after RL reasoning training. Everitt et al. [2017], Skalse et al. [2022] and Gao et al. [2023] describe what a corrupted reward *can* do; we ask what an ordinary optimiser does with one. Work on the group-relative advantage estimator [Yang et al., 2026, Zeng et al., 2025] is orthogonal, since our effect survives exact-expectation gradients.

7 Conclusion

RLVR is known to be a filter at its KL-regularised optimum (Proposition 1), and known to be broken by the pattern of a verifier's errors rather than their rate [Egashira et al., 2026]. We identify one property of that pattern and a mechanism for it. Cross-prompt sharing lets a leak collect gradient from every prompt that accepts it while every prompt that does not pushes back, and without sharing that channel is identically zero: per-prompt copies of the same architecture show no dependence on leak breadth. Controls with a structure-blind base show sharing at RL time is not by itself enough, placing the mechanism in two stages. The quantity to audit is therefore not how often a verifier is wrong, nor breadth of acceptance on its own, but whether the false positives it admits are error modes the model already produces the same way across prompts. This is the sense in which Zhang [2026] find selection of pre-existing error modes rather than learned exploitation.

Limitations and use of LLMs. Proposition 2 assumes a coherent kernel (Definition 3) and a constant pass rate; the kernel varies seventeenfold, so the threshold is directional. Exact enumeration requires a two-token response space and the leak is a designated string, so string identity is not an adequate definition of a shared error mode for real systems. We use one or two leak families per condition, three to ten seeds with a heterogeneous $u = 1$ outcome, and a contaminated SFT base. LLMs assisted with experiment design and manuscript preparation; every number comes from the released code.

References

Johannes Ackermann, Michael Noukhovitch, Takashi Ishida, and Masashi Sugiyama. Gradient regularization mitigates reward hacking in reinforcement learning from human feedback and verifiable rewards. In *International Conference on Machine Learning (ICML)*, 2026.

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12248–12267, 2024.

Xin-Qiang Cai, Wei Wang, Feng Liu, Tongliang Liu, Gang Niu, and Masashi Sugiyama. Reinforcement learning with verifiable yet noisy rewards under imperfect verifiers. *arXiv preprint arXiv:2510.00915*, 2025.

Marc Dymetman. Binary rewards and reinforcement learning: Fundamental challenges. *arXiv preprint arXiv:2605.02375*, 2026.

Kazuki Egashira, Mark Vero, Jasper Dekoninck, Florian E. Dorner, Robin Staab, and Martin Vechev. Delay, plateau, or collapse: Evaluating the impact of systematic verification error on RLVR. In *Conference on Language Modeling (COLM)*, 2026. arXiv:2605.02909v2.

Tom Everitt, Victoria Krakovna, Laurent Orseau, Marcus Hutter, and Shane Legg. Reinforcement learning with a corrupted reward channel. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2017.

Jingchu Gai, Guanning Zeng, Huaqing Zhang, and Aditi Raghunathan. Differential smoothing mitigates sharpening and improves LLM reasoning. *arXiv preprint arXiv:2511.19942*, 2025.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, pages 10835–10866, 2023.

Lukas Helff, Quentin Delfosse, David Steinmann, Ruben Härle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. LLMs gaming verifiers: RLVR can lead to reward hacking. *arXiv preprint arXiv:2604.15149*, 2026.

Yuzhen Huang, Weihao Zeng, Xingshan Zeng, Qi Zhu, and Junxian He. From accuracy to robustness: A study of rule- and model-based verifiers in mathematical reasoning. *arXiv preprint arXiv:2505.22203*, 2025.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Muhammad Khalifa, Zohaib Khan, Omer Tafveez, Hao Peng, and Lu Wang. Countdown-code: A testbed for studying the emergence and generalization of reward hacking in RLVR. *arXiv preprint arXiv:2603.07084*, 2026.

Tomasz Korbak, Ethan Perez, and Christopher L Buckley. RL with KL penalties is better viewed as Bayesian inference. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2022.

Germán Kruszewski, Pierre Erbacher, Jos Rozen, and Marc Dymetman. Whatever remains must be true: Filtering drives reasoning in LLMs, shaping diversity. In *International Conference on Learning Representations (ICLR)*, 2026.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking RLVR's indifference to diversity. *arXiv preprint arXiv:2605.00365*, 2026.

Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural emergent misalignment from reward hacking in production RL. *arXiv preprint arXiv:2511.18397*, 2025.

Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *International Conference on Machine Learning (ICML)*, 2020.

Kei Nishimura-Gasparian, Robert McCarthy, and David Lindner. Towards understanding specification gaming in reasoning models. *arXiv preprint arXiv:2605.02269*, 2026.

Andreas Plesner, Francisco Guzmán, and Anish Athalye. An imperfect verifier is good enough: Learning with noisy rewards. *arXiv preprint arXiv:2604.07666*, 2026.

Ali Rad, Khashayar Filom, Darioush Keivan, Peyman Mohajerin Esfahani, and Ehsan Kamalinejad. Rate or fate? RLV$^{\varepsilon}$R: Reinforcement learning with verifiable noisy rewards. *arXiv preprint arXiv:2601.04411*, 2026.

Jaideep Ray. Before the model learns the bug: Fuzzing RLVR verifiers. *arXiv preprint arXiv:2606.01066*, 2026.

Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. Spurious rewards: Rethinking training signals in RLVR. *arXiv preprint arXiv:2506.10947*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. *arXiv preprint arXiv:2209.13085*, 2022.

Jingkang Wang, Yang Liu, and Bo Li. Reinforcement learning with perturbed rewards. In *AAAI Conference on Artificial Intelligence*, 2020.

Zhangchen Xu, Yuetai Li, Fengqing Jiang, Bhaskar Ramasubramanian, Luyao Niu, Bill Yuchen Lin, and Radha Poovendran. TinyV: Reducing false negatives in verification improves RL for LLM reasoning. *arXiv preprint arXiv:2505.14625*, 2025.

Fengkai Yang, Zherui Chen, Xiaohan Wang, Xiaodong Lu, Jiajun Chai, Guojun Yin, Wei Lin, Shuai Ma, Fuzhen Zhuang, Deqing Wang, Yaodong Yang, Jianxin Li, and Yikun Ban. Your group-relative advantage is biased. *arXiv preprint arXiv:2601.08521*, 2026.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Youliang Yuan, Qiuyang Mang, Jingbang Chen, Hong Wan, Xiaoyuan Liu, Junjielong Xu, Jen-tse Huang, Wenxuan Wang, Wenxiang Jiao, and Pinjia He. Curing miracle steps in LLM mathematical reasoning with rubric rewards. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Guanning Zeng, Zhaoyi Zhou, Daman Arora, and Andrea Zanette. Shrinking the variance: Shrinkage baselines for reinforcement learning with verifiable rewards. *arXiv preprint arXiv:2511.03710*, 2025.

Chuyifei Zhang. When the reward suite is leaky: A preregistered causal contrast of natural verifier false positives in RLVR. *arXiv preprint arXiv:2607.11022*, 2026.

Yulai Zhao, Haolin Liu, Dian Yu, Sunyuan Kung, Meijia Chen, Haitao Mi, and Dong Yu. One token to fool LLM-as-a-judge. *arXiv preprint arXiv:2507.08794*, 2025.

A Proofs

A.1 Proposition 1

Fix x. The objective $\sum_y \pi(y) r(x,y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\mathrm{ref}}(y)}$ over the simplex is strictly concave for $\beta > 0$; setting the derivative of the Lagrangian to zero gives $\pi^\star_\beta(y \mid x) = \pi_{\mathrm{ref}}(y \mid x) e^{r(x,y)/\beta} / Z_\beta(x)$ with $Z_\beta(x) = \sum_{y'} \pi_{\mathrm{ref}}(y' \mid x) e^{r(x,y')/\beta}$. For y, y' with $r(x,y) = r(x,y')$ the normaliser and the exponential factor cancel in the ratio. Since $r \geq c$ we have $\mathcal{H}(x) \subseteq \mathcal{A}(x)$ and $r \equiv 1$ on $\mathcal{A}(x)$, so

$$\frac{\pi^\star_\beta(\mathcal{H}(x) \mid x)}{\pi^\star_\beta(\mathcal{A}(x) \mid x)} = \frac{e^{1/\beta} \pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)/Z_\beta(x)}{e^{1/\beta} \pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)/Z_\beta(x)} = \frac{\pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)}{\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)}$$

for every x, hence ρ and Λ (the latter whenever $\pi(\mathcal{H}(x) \mid x) > 0$) are unchanged. For a multiplicative update $\pi_{t+1}(y \mid x) \propto \pi_t(y \mid x) e^{\eta f(r(x,y))}$ the same cancellation applies at every step.

Excursion of the pooled statistic. Write $a_x = \pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)$, $\rho_x = \pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)/a_x$ and $s = e^{1/\beta}$. Then $\pi^\star_\beta(\mathcal{A}(x) \mid x) = s\,a_x/(1 - a_x + s\,a_x)$, and the pooled statistic is $P(s) = \sum_x \rho_x w_x(s) / \sum_x w_x(s)$ with $w_x(s) = s a_x/(1 - a_x + s a_x)$. At $s = 1$ (no RL) $P = \sum_x a_x \rho_x / \sum_x a_x$; as $s \to \infty$ every $w_x \to 1$ and $P \to \frac{1}{N}\sum_x \rho_x = \rho(\pi_{\mathrm{ref}})$. The total excursion is therefore

$$P(\infty) - P(1) \;=\; \frac{1}{N}\sum_x \rho_x \;-\; \frac{\sum_x a_x \rho_x}{\sum_x a_x} \;=\; -\frac{\mathrm{Cov}_x(a_x, \rho_x)}{\mathbb{E}_x[a_x]},$$

which is positive whenever pass rate and hack fraction are negatively correlated across prompts. The pooled statistic therefore drifts upward under pure filtering and is invariant only in the degenerate case $\rho_x \equiv$ const. It is not an estimator of ρ and we do not call it unbiased; it is a deterministic function of the filtering path that happens to converge to ρ in the strong-RL limit.

A.2 Lemma 1

$\nabla_\theta J = \frac{1}{N}\sum_{x'}\sum_{y'} \pi_\theta(y' \mid x') r(x',y') \nabla_\theta \log \pi_\theta(y' \mid x') = \frac{1}{N}\sum_{x'} \mathbb{E}_{y'}[r(x',y') \nabla_\theta \log \pi_\theta(y' \mid x')]$. Since $\mathbb{E}_{y'}[\nabla_\theta \log \pi_\theta(y' \mid x')] = 0$, the expectation equals $\mathrm{Cov}_{y'}(r, \nabla_\theta \log \pi_\theta)$, understood componentwise. For a fixed vector $v = \nabla_\theta \log \pi_\theta(y \mid x)$, $\langle v, \mathrm{Cov}(r, g)\rangle = \mathbb{E}[r\langle v, g\rangle] - \mathbb{E}[r]\mathbb{E}[\langle v, g\rangle] = \mathrm{Cov}(r, \langle v, g\rangle)$, so the inner product passes inside. □

A.3 Corollary 1

With $\theta = (\theta_x)$ and $\pi_\theta(\cdot \mid x)$ a function of θ_x only, $\nabla_{\theta_{x'}} \log \pi_\theta(y \mid x) = 0$ for $x' \neq x$, so every off-diagonal kernel entry is a sum of products with a zero factor. Moreover $\nabla_{\theta_x} J = \frac{1}{N}\sum_y \pi_\theta(y \mid x)(r(x,y) - V(x)) \nabla_{\theta_x} \log \pi_\theta(y \mid x)$ involves only θ_x, so the flow restricted to block x is autonomous and depends on the verifier only through $r(x, \cdot)$. □

Rich-get-richer under vanilla policy gradient. For a single prompt with free logits θ_y and $J = \sum_y \pi_y r_y$, $\partial J/\partial \theta_z = \pi_z(r_z - J)$, so for two responses with $r_1 = r_2 = 1$, $\frac{d}{dt}\log(\pi_1/\pi_2) = \dot\theta_1 - \dot\theta_2 = (\pi_1 - \pi_2)(1 - J)$. The odds between equally rewarded responses are therefore *not* preserved by vanilla policy gradient, even with no parameter sharing; they move toward whichever response already has more mass, at a rate that vanishes only as $J \to 1$. This is the prompt-local drift referred to in Section 3; it is independent of u by Corollary 1.

A.4 Proposition 2

Two parts of the kernel drop out before Definition 3 is applied: any component of $K_\theta((x,h),(x',y'))$ that does not depend on y' contributes zero covariance in (1), and any component that does not depend on h is common to the leak and to the correct answer and cancels in the within-prompt log-odds. Substituting the definition, the covariance at x' is

$$\begin{aligned}\mathrm{Cov}_{y'}[r(x',y'),K_\theta((x,h),(x',y'))] &= \kappa\Big(\pi_\theta(h\mid x')r(x',h)-\pi_\theta(h\mid x')V(x')\Big)\\ &= \kappa\,\pi_\theta(h\mid x')\big(r(x',h)-V(x')\big),\end{aligned}$$

using $\mathbb{E}_{y'}[r]=V(x')$, with κ replaced by κ_s in the $x'=x$ term. Since $r(x',h)=\mathbf{1}[x'\in U]$, for $x\in U$,

$$\begin{aligned}\frac{d}{dt}\log\pi_\theta(h\mid x) = \frac{\kappa}{N}\Big[\sum_{x'\in U\setminus\{x\}}\pi_\theta(h\mid x')(1-V(x'))-\sum_{x'\notin U}\pi_\theta(h\mid x')V(x')\Big]\\ +\frac{\kappa_s}{N}\pi_\theta(h\mid x)(1-V(x)).\end{aligned}$$

With $\pi_\theta(h\mid\cdot)\equiv p_h$ and $V\equiv\bar V$ the bracket is $p_h[(m-1)(1-\bar V)-(N-m)\bar V]=p_h[N(u-\bar V)-(1-\bar V)]$, giving $\kappa p_h(u-\bar V)+\frac{p_h(1-\bar V)}{N}(\kappa_s-\kappa)$. The identical computation for g_x with scales $(\kappa_g,\kappa_{s,g})$ and universality u_g gives $\kappa_g p_g(u_g-\bar V)+\frac{p_g(1-\bar V)}{N}(\kappa_{s,g}-\kappa_g)$; subtracting yields (3). □

Validity of the leading-order threshold. The $O(1/N)$ terms are negligible when $N\kappa/\kappa_s\gg 1$. We measure κ/κ_s as the mean cross-prompt gradient cosine for the leak, and it is not constant: +0.29 at $u=1$ gives $N\kappa/\kappa_s\approx 18$, but +0.02 at $u=0.016$ gives $N\kappa/\kappa_s\approx 1.1$, at which point the correction is of the same order as the leading term. The leading-order threshold is therefore an argument about the high-universality end only; the suppression we observe at low u is an empirical finding, not a prediction of (3). Proposition 2 also treats κ as a constant while we measure it growing roughly seventeenfold with u, so the observed growth of the cross-prompt drift is jointly attributable to $\kappa(u)$ and to $(u-\bar V)$ and the proposition does not separate them. Both are reasons we do not read a numerical boundary off the experiments.

Reading the threshold. $\bar V$ is the policy's *verifier* pass rate, observable during training without any ground truth. A leak is amplified when it is accepted more often than the policy currently passes. The statement is directional and first-order: it identifies the sign of the leading-order drift and the quantity controlling it, not an exact boundary in a model whose kernel is only approximately coherent.

B Environment, model and training details

Task. Prompts are all (a,b) with $a,b\in\{2,\dots,9\}$ ($N=64$); the correct response is the two-digit decimal string of ab (all products lie in $[4,81]$). The response space is all 100 two-digit strings. Reducing the operand set to $\{2,\dots,5\}$ gives $N=16$ for the prompt-set-size sweep, which uses three seeds and the learning-rate grid $\{3\times10^{-4},10^{-3},3\times10^{-3}\}$ shared by both arms. The correct answers are not perfectly prompt-specific (2×6 and 3×4 both give 12), so they carry a universality of their own, $u_g=0.037$. Under Definition 3 what matters is measured coherence rather than string identity, and we find the correct responses to be near-incoherent across prompts (+0.03) while the leak is not. u_g is small and non-zero, and we report it rather than assume it away.

Leak assignment. Leak strings are drawn without replacement from the 69 two-digit strings that are not the correct answer of any prompt, so a leak is never a correct answer anywhere and every response is unambiguously classified. Prompts are partitioned into N/m blocks; each block receives k leak strings shared by all its prompts. The verifier's static false-positive rate is exactly $k/99$ in every condition, independent of m.

Blocks are contiguous in the default configuration, and with prompts enumerated first-operand-major a contiguous block of size $m\geq 8$ is exactly a set of first operands, a property the transformer can read off a single input token. That would confound universality with how simple the prompt-to-leak map is to represent, so we run the whole sweep a second time with block membership given by a random permutation of prompts and leak strings drawn independently per (m,k,seed) rather than nested across m. Both versions appear in Figures 1 and 3.

What the base model does and does not control. The SFT target puts identical hack mass on every prompt in every condition, but the realised base model is a shared-weight transformer that fits a universal leak map slightly better than an idiosyncratic one, so the measured base hack fraction still varies by 0.015 across universality conditions. We therefore report *drifts* from each run's own base model, $\Delta\rho$ and $\Delta\Lambda$, which Proposition 1 makes exactly zero under the filter baseline and which are immune to that residual mismatch. Absolute ρ is shown alongside the base curve so both are visible.

In the universality×error-rate factorial an additional correction is needed. The uniform component of the SFT target would otherwise place mass on each of the k leak strings, so base hack mass would grow with k and the two factors would not be separable. For that experiment the uniform component is spread only over responses that are neither the correct answer nor a leak, which makes total base hack mass exactly ε for every k. The measured base *fraction* still varies, mean $\rho = 0.128$ with spread 0.054 across runs, because diverting the uniform floor also changes the mass on the correct answer; it is the mass, not the fraction, that the design holds fixed. This is the one place where the SFT target differs between experiments, and it is the version the factorial requires.

Base model. A decoder-only transformer with 2 layers, model dimension 64, 4 heads and a 10-way output head over digits (101,706 parameters) is trained for 600 Adam steps at 3×10^{-3} on the exact target $q(y \mid x) = (1-\varepsilon)[\lambda_x \mathbf{1}[y = g_x] + (1-\lambda_x)\mathrm{Unif}(\mathcal{Y})] + \varepsilon\,\mathrm{Unif}(\mathcal{H}(x))$ with $\varepsilon = 0.03$. Competence levels λ_x are an evenly spaced grid on $[0, 0.9]$ assigned by a fixed permutation with a seed independent of the block structure, so difficulty and universality are uncorrelated by construction. Because the loss is against an exact target rather than samples, the resulting base distribution is matched across universality conditions without any per-condition calibration.

Tabular policy. Free parameters $w_1 \in \mathbb{R}^{N\times 10}$ and $w_2 \in \mathbb{R}^{N\times 10\times 10}$ with $\pi(y \mid x) = \mathrm{softmax}(w_1[x])_{y_1} \cdot \mathrm{softmax}(w_2[x, y_1])_{y_2}$. This factorisation is complete: 10 free first-token logits plus 10×10 free second-token logits parameterise the entire 100-dimensional simplex for each prompt. Given the base transformer's exact log-probabilities we set $w_1 = \log \sum_{y_2} \pi(y \mid x)$ and $w_2 = \log \pi(y \mid x) - w_1$, which reproduces the base distribution to within 10^{-4} in max-norm (asserted at run time).

RL. The default objective is the exact-expectation policy gradient with the exact mean baseline, $\nabla_\theta J = \frac{1}{N}\sum_x \sum_y \pi_\theta(y \mid x)\,[r(x,y) - V(x)]\,\nabla_\theta \log \pi_\theta(y \mid x)$, implemented as a weighted cross-entropy with detached weights. This is the infinite-group limit of RLOO and Dr. GRPO and removes sampling noise from the comparison. The sampled-GRPO experiment uses group size 8, group-standard-deviation normalisation and one inner epoch. Optimiser Adam (default) or SGD, 400 steps, learning-rate grid $\{3\times10^{-4}, 10^{-3}, 3\times10^{-3}, 10^{-2}, 3\times10^{-2}\}$ for Adam and $\{0.3, 1, 3\}$ for SGD, applied identically to both policy classes in the sweeps where a grid is shared (Table 1; E3 and E5 use a fixed neural/tabular pair, E6 two). The reported operating point is, for each policy class and cell, the smallest learning rate whose final verifier reward is within 10^{-3} of the best, which is what a practitioner without ground truth would select (strict argmax in E6).

The mask-only controls (E9, E9b). One checkpoint per seed serves every universality condition. Its supervised target is contaminated with a shared string h and a per-prompt string s_x at every prompt, in identical proportion in all conditions, so the initial policy, the probability on each leak string and the initial tangent kernel are bit-identical across the sweep; we verify the base probability on h agrees to six decimals. The verifier then accepts h on the m prompts in U and, in E9, s_x on the rest, giving exactly N accepted wrong pairs and a static false-positive rate of exactly $1/(|\mathcal{Y}| - 1)$ in every condition. E9b accepts nothing wrong outside U, so every accepted wrong string is h and its base mass is identical across conditions, at the cost of a static false-positive rate equal to $u/99$ and therefore rising with u. Note also that only the *per-prompt* mass on the accepted string is held fixed in E9b: the aggregate accepted-wrong base mass, and with it base ρ, still scale with u because fewer prompts accept anything wrong at small m. Both that and the rising rate bias E9b's slope downward, which is why the shared-versus-per-prompt comparison is made within the same base rather than across conditions. The rising rate which if anything biases E9b's slope downward given that a higher rate mildly reduces hacking in our factorial. U is nested across m, so adjacent conditions differ only by which prompts switched.

The residual gradient in E9 is that the accepted string's base mass rises from 0.0098 to 0.0149: a shared-parameter SFT represents one string demonstrated at every prompt better than N distinct ones. We tried to remove it by reweighting the two families in the supervised target until their realised probabilities matched, iterating a damped fixed point. The mean gap closes to 6×10^{-4} but the per-prompt gap does not, staying near 6×10^{-3} against probabilities of about 1.2×10^{-2}: the shared network cannot represent an arbitrary per-prompt split. Universality and representational ease are therefore not independently manipulable in a shared-parameter model, since sharing is the common cause of both, which is why we bracket the confound with two designs rather than eliminating it in one. E9's and E9b's per-prompt arms responding to that residual gradient is what Corollary 1 permits, since the corollary rules out u-dependence generated by the dynamics, not u-dependence inherited at initialisation.

Per-prompt weight copies (E10). N independent copies of the same transformer, each initialised from the shared base checkpoint and trained on its own prompt alone with the same objective and optimiser, on the three smallest learning rates of the shared arm's grid. Copy x only ever evaluates prompt x, so the copies are independent and training them in one loop is equivalent to training them separately. The matched shared-parameter arm it is compared against is E1 restricted to the same four universality conditions and the same ten seeds, with slopes averaged per seed, which is why its slope (+0.0210) differs slightly from the seven-condition figure quoted in Section 5 (+0.0177).

Full specification. Prompts are serialised as the token sequence $[\text{BOS}, a, b, y_1]$ over a vocabulary of 12 symbols (10 digits plus BOS and a padding symbol), with learned token and learned absolute positional embeddings, a causal mask, pre-norm blocks, 4 heads, feed-forward width $4d = 256$ with GELU, final `LayerNorm`, and a 10-way output head shared across the two response positions. Embeddings are initialised $\mathcal{N}(0, 0.02^2)$ and the remaining parameters use PyTorch defaults. The correct response for prompt (a, b) is the decimal string of the *product* $a \times b$, zero-padded to two digits, so 2×3 has correct response `06`; all products lie in $[4, 81]$ and the response space is every two-digit string `00`–`99`. Optimisation is full-batch over all N prompts with no minibatching and no gradient clipping. Adam uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ and zero weight decay; SGD uses no momentum. A run's seed simultaneously sets the transformer initialisation, the leak-string draw and, in E4, the rollout sampling; the competence permutation is drawn from a separate fixed seed so difficulty is never correlated with the leak structure.

The default objective is exact-expectation policy gradient, $\nabla_\theta J = \frac{1}{N} \sum_x \sum_y \pi_\theta(y \mid x)[r(x, y) - V(x)]\nabla_\theta \log \pi_\theta(y \mid x)$, with $V(x) = \sum_y \pi_\theta(y \mid x) r(x, y)$ the exact mean baseline. E4 replaces it with sampled GRPO: $G = 8$ responses are drawn per prompt at temperature 1 from the full enumerated distribution, advantages are $(r_i - \text{mean})/(\text{std} + 10^{-4})$ within each group, groups whose reward standard deviation is zero contribute zero advantage and hence no gradient, and one inner epoch is taken per batch with no importance ratio, no clipping and no reference-KL term, so the estimator is group-normalised REINFORCE rather than a ratio-clipped PPO objective. E5 adds $-\beta \, \text{KL}(\pi_\theta(\cdot \mid x) \| \pi_{\text{ref}}(\cdot \mid x))$ computed exactly in closed form against the frozen base, not estimated from samples.

Measurement. ρ, Λ and true accuracy are computed from the exact enumerated distribution $\pi_\theta(\cdot \mid x)$ over all 100 responses at all N prompts, at every logged step; they are not Monte-Carlo estimates, and the only stochasticity is over seeds (transformer initialisation, leak-string draw, and rollout sampling in the GRPO condition). Two probes are exceptions: the gradient decomposition and the coherence measurement average over a fixed random subsample of 24 and 12 prompts respectively, for cost.

Per-prompt log-odds are clipped to ± 20 before averaging, so a prompt whose leak mass has collapsed cannot dominate Λ. In the headline Adam sweep this binds on a median of 1.6% of prompts (worst case 18.8%). It binds far harder where the policy collapses outright, up to 81% under SGD at $u = 1$, but neither the SGD nor the GRPO sweep contributes a reported Λ, and on the sweeps that do the ordering of every reported series is unchanged at clipping thresholds of 20, 40, 60 and 200. The final metric of every run is recomputed in float64 from stored per-prompt masses, so the 10^{-8} floor used during online logging does not enter any reported number.

The two null-model checks in Section 5 agree to the last bit by construction: 400 steps of mirror descent at $\eta = 0.05$ reach exactly $\pi_0 e^{20r}$, which is the $\beta = 0.05$ endpoint of the KL grid. They are one computation checked two ways, not two independent checks.

Per-experiment protocol. Seeds, learning-rate grids and the operating-point rule vary between experiments; Table 1 records what was actually run.

	sweep	seeds	learning rates	notes
E1	universality, contiguous blocks	10	5, shared by both arms	headline sweep
E2	universality, permuted blocks	3	4, shared by both arms	base spread 0.0011
E3	universality $\times$ error rate	3	1 pair (10^{-3} / 3×10^{-2})	clean SFT floor
E4	sampled GRPO	10	3, shared by both arms	$G = 8$, std normalisation
E5	KL sweep	3	1 pair (10^{-3} / 3×10^{-2})	6 values of β
E6	verifier hardening	3	2	strict argmax on verifier reward
E7	prompt-set size $N \in \{16, 64\}$	3	3, shared by both arms	tests u versus m
E8	plain SGD	10	3, shared by both arms	robustness
E9	mask-only, constant error rate	10	5, shared by both arms	one frozen base per seed
E9b	mask-only, constant leak mass	5	5, shared by both arms	one frozen base per seed
E10	per-prompt weight copies	10	3	same architecture as the shared arm

Table 1: What was run in each experiment. Where a grid is shared by both arms the operating point is the smallest learning rate within 10^{-3} of the best verifier reward, chosen independently for each arm.

C Additional results

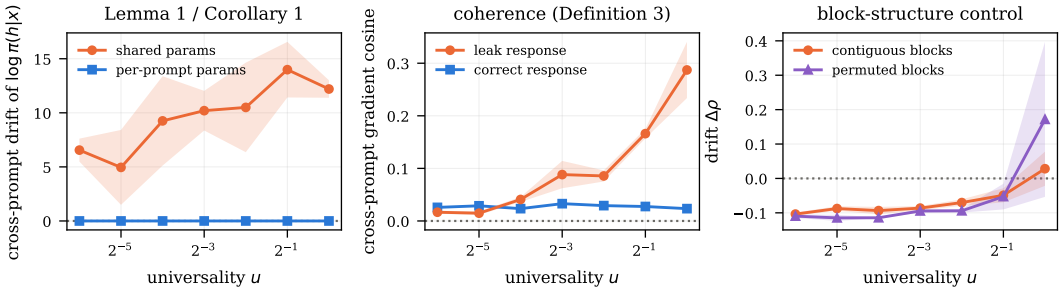


Figure 3: Mechanism. Left: the cross-prompt component of the exact drift of $\log \pi_\theta(h \mid x)$ at the base model (Lemma 1), identically zero for per-prompt parameters as Corollary 1 requires. Centre: mean pairwise cosine between per-prompt gradients, the coherence of Definition 3, measured rather than assumed. Right: contiguous versus randomised block membership.

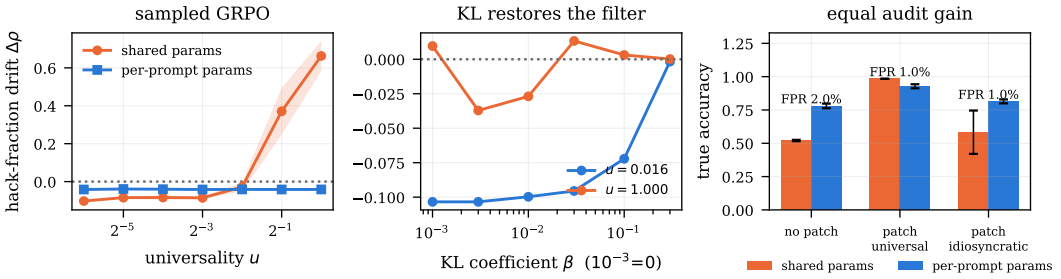


Figure 4: Practice. Left: sampled GRPO reproduces the pattern. Centre: a KL penalty pulls the policy back toward the filter baseline; the low-u side is resolved, the $u = 1$ side is within noise. Right: two verifier patches that improve the measured false-positive rate identically buy very different true accuracy.

Verifier hardening (E6). We give a verifier two leak families, one universal string plus one idiosyncratic string per prompt, and compare two patches that remove exactly the same number of accepted wrong (prompt, response) pairs and so improve any static audit identically. Removing the universal leak leaves the policy at 0.984 true accuracy; removing every idiosyncratic leak instead leaves it at 0.583, a gap of 40 ± 16 points (3 seeds) from patches an audit rates as equally valuable (unpatched: 0.521). The per-prompt policy, which by Corollary 1 cannot respond to universality at all, shows a gap of only 11 points between the same two verifiers. That residue is what the base model already carried, since the SFT stage is itself shared-parameter (base hack fractions 0.064 and 0.094). The remaining 29 points are produced by RL.

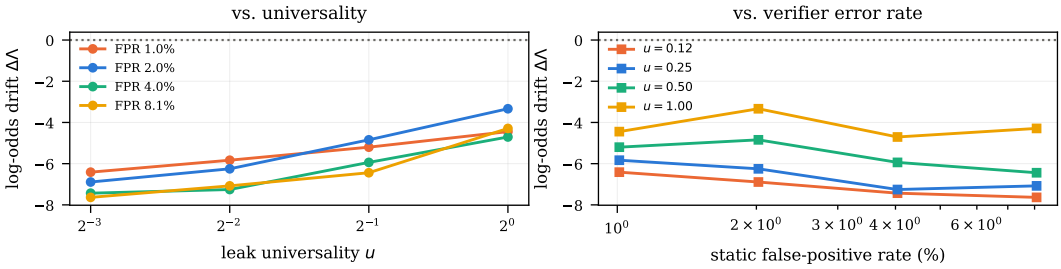


Figure 5: In this environment universality moves the drift and the verifier's error rate does not. Left: log-odds drift against universality, one line per false-positive rate. Right: the same data against the static false-positive rate, one line per universality, common y-scale. Varying accepted wrong responses per prompt from 1–8 moves the false-positive rate over 1.0–8.1% and mildly *reduces* hacking; universality moves the drift monotonically and much further.

Prompt-set size (E7). At fixed u the analysis predicts no dependence on the number of prompts. Repeating the sweep at $N = 16$ and $N = 64$ is consistent with that: at $u = 1$ the two agree (-0.004 ± 0.084 versus $+0.010 \pm 0.045$), whereas at the same block size $m = 16$ they sit at -0.079 against -0.004, a gap the three seeds do not resolve. Across the 5 matched values of u the largest gap is 0.065.

Figure 6 collects the four robustness views that do not fit in the main text: the universality effect across the whole Adam learning-rate grid rather than at the single reward-selected operating point, hack-fraction trajectories at the two extreme universality conditions for both policy classes, the sweep repeated at two prompt-set sizes, and the sweep under plain SGD.

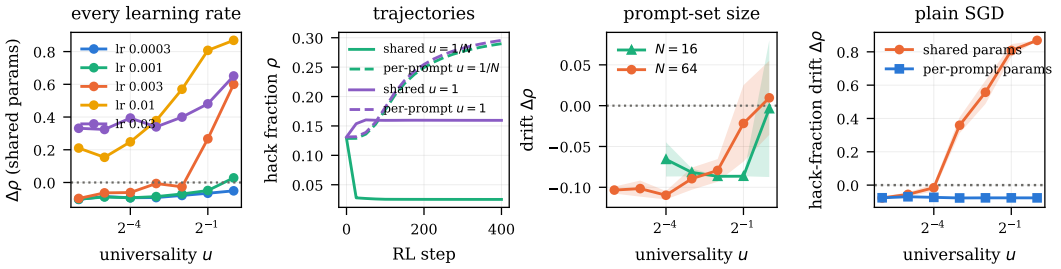


Figure 6: Left: the universality effect across the whole Adam learning-rate grid, rather than at the reward-selected operating point. Centre left: hack-fraction trajectories for the extreme universality conditions, both policy classes. Centre right: the sweep repeated at two prompt-set sizes, plotted against u. Right: the same universality sweep under plain SGD.

D Reproducibility

All code, raw result files and figure scripts are included with the submission. A single machine with two CPU cores reproduces every experiment in the paper; no GPU is used. `src/env.py` defines the task and verifier, `src/policies.py` the two policy classes, `src/rlvr.py` the objectives and exact measurement, `src/theory.py` the numerical verification of Proposition 1 and Lemma 1,

and `src/exp*.py` the experiments. Seeds 0–4 of E1 were produced by an earlier revision whose configuration records carry fewer fields. Re-running a cell from that half under the current revision reproduces it to 10^{-8} and the stored base models are bit-identical, so the halves are pooled. One cell of E1 (the $m = 1$ condition) was run beyond the reported seed budget while a larger budget was being attempted; `src/figures.py` caps every E1, E4, E8, E9, E9b and E10 load at its reported seed budget and drops any seed not present at every universality condition, so the reported design is balanced at ten seeds for every universality condition rather than giving a few conditions more seeds than the rest. `src/figures.py` regenerates every figure and the macro file holding every measured number quoted in the text; the hand-entered constants are the step count, the contamination rate and the GRPO group size, which are settings rather than results, together with a small number of measured figures quoted in the text rather than through a macro: the clipping rates, $N\kappa/\kappa_s$, the seventeenfold growth of κ and its peak at $u = 0.5$, and the leave-one-seed-out drift of -0.017, all recomputed from the shipped result files, and the three probability gaps from the E9 reweighting attempt above, which come from a diagnostic run whose output is not shipped.